HOSTFI Telegram
Automation Bot System

Full Project Plan · Architecture · Build Roadmap · Feature Specification

Document Type	Project System Plan & Architecture
Product	HOSTFI Community Telegram Bot
Version	v1.0 — Initial Release Scope
Status	Ready for Development
Prepared By	Bot Development Lead

This document outlines the complete technical plan, architecture, and build strategy for the HOSTFI Telegram Automation Bot. It is intended for developers, stakeholders, and team members who need a full understanding of the system.

■ TABLE OF CONTENTS

01 EXECUTIVE SUMMARY

The HOSTFI Telegram Automation Bot is a comprehensive, AI-powered community management and engagement system designed specifically for the HOSTFI crypto-fintech platform's Telegram community. The bot acts as a 24/7 intelligent assistant that simultaneously manages the community, answers user questions accurately, delivers live market data, and creates an engaging experience — all without human intervention.

Unlike a simple command bot, this system is architected as a multi-surface automation layer with six distinct functional modules. It leverages a RAG (Retrieval Augmented Generation) AI engine to ensure all answers about HOSTFI are grounded in verified, up-to-date information — eliminating the risk of AI-generated misinformation in a financial context.

6	5 Weeks	RAG AI	~$5/mo
Functional Modules	Build Timeline	Verified Answers Only	Running Cost

02 WHAT IS HOSTFI?

HOSTFI is a crypto-fintech mobile application that provides users with a seamless suite of financial services spanning both traditional fiat currency and digital assets.

Feature	Description	Type
Buy & Sell Crypto	Purchase or liquidate digital assets instantly	Core
Fiat Deposit/Withdrawal	Deposit and withdraw local currency (e.g. NGN)	Core
Crypto Swaps	Exchange one digital asset for another seamlessly	Core
Virtual Cards	Spend crypto/fiat via virtual Visa/Mastercard cards	Premium
Wallet Management	Multi-asset digital wallet with transaction history	Core

03 WHY THIS BOT EXISTS

Telegram is one of the primary community channels for crypto-fintech platforms in emerging markets, especially across West Africa. As a community grows, three critical problems emerge:

■ **Support Overload**	Admins get flooded with the same repetitive questions — "How do I fund my card?" "What are the swap fees?" — pulling human resources away from high-value tasks.
■ **Spam & Scams**	Crypto communities are prime targets for scammers posting fake wallet addresses, phishing links, and impersonating support staff. Manual moderation at scale is impossible.

■ **Poor Engagement**	Without automation, announcements are inconsistent, new members feel ignored, and community activity declines — hurting app adoption and brand trust.

The HOSTFI Bot solves all three problems simultaneously, acting as a force multiplier for the community team.

04 SYSTEM OVERVIEW & ARCHITECTURE

4.1 Three Interaction Surfaces

The bot is not a single-purpose tool. It operates across three distinct surfaces, each with different user roles, permissions, and behaviors:

Surface	Who Uses It	Purpose
■■ **Group / Community Mode**	All community members	Moderation, announcements, market data, onboa
■ **Personal DM Mode**	Individual users	AI support assistant, ticket creation, price alerts
■ **Admin / Ops Mode**	Bot admins & HOSTFI team	Broadcasts, reports, user management, monitorin

4.2 High-Level Architecture

TELEGRAM	User sends message/command in group or DM	**INPUT**
FASTAPI	Webhook handler receives and validates the update	**GATEWAY**
ROUTER	Identifies command type, checks user permissions	**LOGIC**
MODULES	Routes to correct module (Moderation / AI / Market)	**PROCESSING**
AI ENGINE	RAG query → ChromaDB → Groq → Verified answer	**AI LAYER**
DATABASE	Supabase stores users, tickets, logs, settings	**STORAGE**
RESPONSE	Bot sends reply back to Telegram	**OUTPUT**

05 TECHNOLOGY STACK

The entire system is built on free and low-cost tools without sacrificing performance or reliability. The estimated total running cost is approximately $5/month after any free tier limits.

Layer	Tool / Service	Purpose	Cost
Bot Framework	python-telegram-bot v20+	Core async bot engine, handles all Telegram events	Free
Language	Python 3.11+	Primary development language	Free
Web Framework	FastAPI	Webhook handler for incoming Telegram updates	Free
AI Completion	Groq API (Llama 3.3 70B)	Generate answers from RAG context chunks	Near-free
Embeddings	sentence-transformers	Convert text to vectors for semantic search	Free
Vector Database	ChromaDB (local)	Store and search knowledge base embeddings	Free
Main Database	Supabase (PostgreSQL)	User data, tickets, logs, settings	Free tier
Cache / Queue	Upstash Redis	Rate limiting, session cache, job queue	Free tier
Market Data	CoinGecko API	Live crypto prices and market stats	Free
Hosting	Railway.app	Deploy and run the bot 24/7	~$5/mo
Monitoring	BetterStack	Uptime monitoring, alerts, log management	Free tier
Task Scheduler	APScheduler	Timed broadcasts, daily digests, auto-tasks	Free

06 MODULE BREAKDOWN

The bot is organized into 6 independent modules. Each module can be developed, tested, and deployed separately, making the project maintainable and scalable.

M1	**Community Management**	Moderation, welcome, anti-spam, admin commands
M2	**AI Support Assistant**	RAG-powered Q&A;, guardrails, escalation
M3	**Live Market Data**	Prices, rates, alerts, daily digest
M4	**Broadcast & Engagement**	Announcements, polls, referrals, XP system
M5	**Support Ticket System**	Private tickets, routing, resolution tracking
M6	**Admin Dashboard**	Ops channel, daily reports, broadcast composer

6.1 Module 1 — Community Management Engine

This is the backbone of the bot's group presence. It handles everything that happens in the HOSTFI Telegram community — from the moment a new member joins to enforcing community rules automatically.

■ **Smart Welcome Flow**	New members receive a personalized welcome message with a CTA card linking them to the HOSTFI app. The welcome includes quick-start buttons and community rules.
■■ **Anti-Spam Engine**	Detects and deletes spam messages using keyword filters, regex pattern matching, and duplicate message detection. Works silently without disrupting conversation flow.
■ **Scam Link Detection**	Automatically scans every message for known phishing domains, fake wallet addresses, and impersonation patterns. Offending messages are deleted instantly with an alert.
■ **Verification Gate**	New members must complete a simple CAPTCHA or quiz question before they can post. This eliminates bot accounts and ensures member quality.
■■ **Warn / Mute / Ban**	Three-tier moderation system. First offense: warning. Second: temporary mute. Third: ban. All actions are logged with timestamps and reasons.
■ **Admin Commands**	/warn, /mute [duration], /ban, /unban, /kick, /pin, /rules, /announce — full admin toolkit usable only by authorized admins.
■■ **Flood Control**	Rate-limiting per user prevents message flooding. Users who post more than X messages in Y seconds are auto-muted temporarily.

■ **Auto-Rules Posting**	Rules are automatically pinned and re-posted on a schedule to keep community guidelines visible to all members.

6.2 Module 2 — AI Support Assistant (RAG Engine)

This is the most technically sophisticated module. It enables the bot to answer questions about HOSTFI intelligently and accurately — without ever making up information. This is achieved through a technique called RAG (Retrieval Augmented Generation).

How It Works (Step by Step):

STEP 1	**Knowledge Base Creation** All public HOSTFI content (website, app store listing, FAQs, social media, manually written guides) is scraped, cleaned, and split into small text chunks.
STEP 2	**Embedding Generation** Each text chunk is converted into a numerical vector (embedding) using sentence-transformers. This allows the system to do semantic/meaning-based search.
STEP 3	**Vector Storage** All embeddings are stored in ChromaDB — a local vector database optimized for fast similarity search.
STEP 4	**Query Processing** When a user asks a question, the question is also converted to an embedding. ChromaDB finds the top 3 most semantically similar chunks from the knowledge base.
STEP 5	**Grounded AI Response** Only those matching chunks are sent to the Groq AI (Llama 3.3 70B). The AI is instructed to ONLY answer using the provided context — no fabrication allowed.
STEP 6	**Guardrails Applied** If no relevant chunks are found (below confidence threshold), the bot responds: 'I'm not sure about that — please contact HOSTFI support directly.'

Safety Guardrails:

- Confidence threshold — if similarity score is too low, the bot refuses to answer and redirects to human support
- Topic boundary — the AI is system-prompted to only discuss HOSTFI-related topics
- Disclaimer on fees/rates — any answer involving financial figures appends: 'Confirm current rates in the app'
- Emergency escalation — keywords like 'hacked', 'lost funds', 'scammed' skip AI entirely and ping a human admin
- No personal advice — the bot never provides investment advice or price predictions

6.3 Module 3 — Live Market Data

Powered entirely by the free CoinGecko API, this module gives the community real-time access to crypto market data without any API key required.

Command	What It Does
`/price [coin]`	Get live price of any cryptocurrency in NGN, USD, USDT
`/rates`	Show current exchange rates relevant to HOSTFI (BTC, ETH, USDT, SOL
`/market`	Show top 10 crypto by market cap with 24h change
`/alert set [coin] [price]`	Subscribe to a price alert — bot DMs you when target is hit
`/alert cancel`	Cancel an existing price alert subscription
`/fear`	Show the current Crypto Fear & Greed Index with interpretation
`/digest`	Auto-posted daily summary: top movers, market sentiment, HOSTFI releva

6.4 Module 4 — Broadcast & Engagement Engine

This module transforms the bot from a reactive tool into a proactive community builder. It drives engagement, rewards loyalty, and keeps the community active between announcements.

■ Broadcast System

• Admins compose rich announcements with photos, buttons, and formatting
• Schedule broadcasts for optimal send time
• Target specific user segments (e.g. all DM users)
• Track delivery and engagement rates

■ Engagement Features

• Community XP system — active members earn points
• Referral tracking — invite leaderboard with rewards
• Weekly polls and community surveys
• Milestone badges for top contributors

6.5 Module 5 — Support Ticket System

When the AI assistant cannot resolve an issue, or when a user explicitly needs human help, the support ticket system takes over. This creates an auditable, manageable pipeline for real human support interactions.

User	Types /support or clicks Support button
Bot	Asks user to briefly describe their issue
System	Creates ticket with unique ID, logs timestamp and user info
Admin Channel	Ticket alert posted to admin ops channel with user details
Admin	Clicks 'Take Ticket' to claim it and opens private thread
Resolution	Admin resolves, closes ticket. User receives confirmation + rating prompt

6.6 Module 6 — Admin Intelligence Dashboard

A private Telegram channel serves as a live ops dashboard for the bot administrators. Everything the bot does is surfaced here in real time — making the team fully aware of community health without logging into any external tool.

Feature	Description
Daily Automated Report	Posted every morning: new members, messages processed, commands used, bans/wa
Real-time Alerts	Instant alerts for: scam detected, high-severity ticket opened, unusual member activity,
Broadcast Composer	Admins type /broadcast in the admin channel and compose rich announcements that g
User Lookup	Admins can query any user's bot history: join date, warns, tickets, commands used
Knowledge Base Updates	Admin command to trigger a re-scrape and re-index of the RAG knowledge base when

07 RAG SYSTEM — HOW THE AI STAYS ACCURATE

RAG stands for Retrieval Augmented Generation. It is the technique that makes the AI support assistant safe and reliable in a financial context. Here is a plain-English explanation of the full system:

■■ The Knowledge Base	Think of this as a filing cabinet. Before the bot goes live, we fill this cabinet with every piece of information about HOSTFI — from their website, app store descriptions, FAQ pages, and manually written support guides. This is your source of truth.
■ Semantic Search (Not Keyword Search)	When a user asks 'How do I top up my HOSTFI card?', the system doesn't just search for the exact words. It understands the meaning of the question and finds the most relevant sections from the knowledge base — even if they use different words.
■ AI as a Summarizer, Not an Inventor	The AI's job is ONLY to take the relevant sections found and turn them into a clear, conversational answer. It cannot go beyond what it was given. If the answer isn't in the knowledge base, it says so and points to human support.
■ Keeping It Updated	Whenever HOSTFI updates their app or website, the knowledge base is re-ingested. This takes about 5 minutes and ensures the bot is always current.

08 SECURITY ARCHITECTURE

Security is non-negotiable for a fintech-adjacent product. Even though the bot does not directly access HOSTFI's backend systems in v1, it still handles user data, community interactions, and directs users toward financial actions.

■ **Webhook Signature Verification**	Every request from Telegram is verified using a secret token. Forged requests are rejected immediately.
■ **Input Sanitization**	All user input is sanitized before being processed or stored. Prevents injection attacks.
■ **Rate Limiting**	All commands are rate-limited per user to prevent abuse and denial-of-service.
■ **Admin Permission System**	Admin commands require verified admin Telegram IDs. IDs are stored securely in environment variables, not code.
■ **No Sensitive Data Stored**	The bot never stores passwords, private keys, or financial account details. Only Telegram user IDs and bot-specific data.
■ **Audit Logging**	All moderation actions (warns, bans, mutes) and admin commands are logged with timestamps.
■ **Scam Pattern Database**	Maintained list of known scam wallet addresses, phishing domains, and impersonation patterns. Updated regularly.
■ **Environment Variable Config**	All API keys, tokens, and secrets are stored in environment variables — never hardcoded in source code.

09 PROJECT FOLDER STRUCTURE

The project follows a clean, modular structure. Each module lives in its own folder and can be developed independently. This makes the codebase maintainable, testable, and easy for new developers to navigate.

```
hostfi-bot/
■
■■■ main.py                    # Entry point — starts the bot and webhook
■■■ config.py                  # All settings, env vars, constants
■■■ requirements.txt           # Python dependencies
■■■ .env                       # Secret keys (never committed to git)
■■■ railway.toml               # Deployment config for Railway.app
■
■■■ bot/
■   ■■■ __init__.py
■   ■■■ handlers/
■   ■   ■■■ community.py        # M1: Welcome, join/leave events
■   ■   ■■■ moderation.py       # M1: Warn, mute, ban, flood control
■   ■   ■■■ admin.py            # M6: Admin commands and permissions
■   ■   ■■■ support.py          # M2: AI assistant query handler
■   ■   ■■■ market.py           # M3: Price, rates, alert commands
■   ■   ■■■ tickets.py          # M5: Ticket creation and management
■   ■   ■■■ broadcast.py        # M4: Broadcast and poll commands
■   ■
■   ■■■ filters/
■   ■   ■■■ spam_filter.py      # Keyword/link/duplicate detection
■   ■   ■■■ scam_filter.py      # Scam wallet and phishing detection
■   ■
■   ■■■ utils/
■       ■■■ permissions.py     # Admin role verification
■       ■■■ formatter.py       # Message formatting helpers
■       ■■■ keyboards.py       # Telegram inline keyboard builders
■
■■■ rag/
■   ■■■ ingestion.py           # Scrape + chunk + embed knowledge base
■   ■■■ retriever.py           # Query ChromaDB for relevant chunks
■   ■■■ ai_engine.py           # Send context + query to Groq API
■   ■■■ guardrails.py          # Confidence checks, topic boundaries
■   ■■■ knowledge_base/        # Raw source documents stored here
■       ■■■ hostfi_website.txt
■       ■■■ hostfi_faq.txt
■       ■■■ manual_guides.txt
■
■■■ scheduler/
■   ■■■ tasks.py               # APScheduler jobs: digest, alerts, reports
■
■■■ database/
■   ■■■ models.py              # Supabase table definitions
■   ■■■ users.py               # User CRUD operations
■   ■■■ tickets.py             # Ticket CRUD operations
■   ■■■ logs.py                # Audit log operations
■
■■■ tests/
    ■■■ test_moderation.py
    ■■■ test_rag.py
    ■■■ test_market.py
```

10 BUILD ROADMAP (PHASE BY PHASE)

The project is broken into 5 weekly phases. Each phase has a clear deliverable that can be tested and demonstrated independently before the next phase begins.

Phase 1	Week 1	**Community Management Core** • Set up project scaffold, Railway deployment, bot token • Implement welcome flow with onboarding card and CTA buttons • Build anti-spam engine (keyword filter, link filter, flood control) • Implement verification gate (CAPTCHA/quiz for new members) • Deploy warn/mute/ban/kick admin command system • Test moderation pipeline end-to-end in a test group
Phase 2	Week 2	**RAG Knowledge Base & AI Assistant** • Scrape and clean all public HOSTFI content • Build ingestion pipeline: chunk → embed → store in ChromaDB • Build retriever: semantic search against knowledge base • Integrate Groq API with strict system prompt and guardrails • Implement confidence threshold and escalation logic • Test with 50 sample user questions, validate accuracy
Phase 3	Week 3	**Market Data & Price Alerts** • Integrate CoinGecko API for live price data • Build /price, /rates, /market, /fear commands • Implement price alert subscription system with Redis storage • Build daily market digest auto-poster with APScheduler • Test alert delivery and scheduler reliability
Phase 4	Week 4	**Broadcast Engine, Tickets & Engagement** • Build admin broadcast composer with scheduling • Implement support ticket creation, routing, and resolution flow • Build community XP system and referral tracking • Implement poll/survey functionality • Build admin intelligence dashboard in private ops channel • Wire all daily/weekly automated reports
Phase 5	Week 5	**Polish, Security Hardening & Launch** • Full security audit: rate limits, input sanitization, permission checks • Load testing: simulate 100+ concurrent users • Build monitoring and alerting via BetterStack • Write admin user guide and knowledge base update SOP • Soft launch in test community, gather feedback • Full production deployment and go-live

11 KPIs & SUCCESS METRICS

These are the metrics that will demonstrate the bot's value to HOSTFI and justify deeper integration:

KPI	Target (Month 1)	Why It Matters
AI queries answered without escalation	> 80%	Shows AI accuracy and reduces human support load
Spam/scam messages blocked	100% detection rate	Protects community reputation and user trust
New member verification completion	> 70%	Filters bots, ensures quality community members
Support ticket resolution time	< 4 hours avg	Benchmarks support efficiency
Daily market digest engagement	> 30% view rate	Measures community activity and bot relevance
Price alert subscriptions	> 50 active users	Shows engagement depth beyond commands
Bot uptime	> 99.5%	Reliability benchmark for a fintech product

12 FUTURE ROADMAP (POST API ACCESS)

Once integrated with the HOSTFI app backend (Phase 2 — post employment/partnership), the following features become immediately unlockable:

■ **Live Balance Checks**
/balance shows user's live wallet balances directly from Hostfi API

■ **Quick Swap Initiation**
Initiate a crypto swap from Telegram, confirm and execute in-app

■ **Transaction Alerts**
Real-time DM notifications when deposits, withdrawals, or card charges occur

■ **Card Management**
Check virtual card status, limits, and recent transactions via bot

■ **Deposit Addresses**
Request a deposit address for any supported asset directly in DMs

■ **Deep Account Linking**
Full OAuth integration linking Telegram identity to Hostfi account securely

■ **Personalized Dashboard**
/dashboard shows user's portfolio summary in a clean Telegram message

■ **P2P Marketplace**
Browse and initiate P2P trades directly through bot commands

13 GLOSSARY — KEY TERMS EXPLAINED

For anyone new to the technical concepts used in this document:

RAG	Retrieval Augmented Generation. A technique where AI answers are grounded in retrieved documents rather than pure model memory, preventing hallucination.
Embedding	A numerical vector representation of text. Allows semantic/meaning-based comparison of text instead of just keyword matching.
ChromaDB	An open-source vector database. Stores embeddings and enables fast similarity search to find relevant knowledge base chunks.
Webhook	A method where Telegram sends updates to our server in real-time whenever something happens (message, join, etc.). More efficient than polling.
FastAPI	A modern Python web framework used to receive and handle incoming Telegram webhook updates.
APScheduler	Advanced Python Scheduler — a library for running tasks on a schedule (e.g. post digest every day at 9am).
Redis	An in-memory data store used for caching, rate limiting, and storing temporary session data (like price alert subscriptions).
Supabase	An open-source Firebase alternative. Provides a PostgreSQL database, REST API, and real-time features — all with a generous free tier.
Groq	An AI inference platform that runs open-source models (Llama 3.3 70B) at very high speed and extremely low cost — near free for moderate usage.
Rate Limiting	Restricting how many times a user can use a command in a given time window. Prevents abuse and server overload.
Confidence Threshold	In the RAG system, the minimum similarity score between a question and knowledge base chunks needed before the AI will attempt to answer.
Inline Keyboard	Telegram buttons that appear beneath messages. Used for interactive flows like ticket creation, verification, and command shortcuts.

Built with precision. Designed for scale. Ready to grow with HOSTFI.

This document is a living blueprint. As the HOSTFI platform evolves, so does the bot.